

ACTIVIDAD 1: SISTEMA DE GESTION DE VEHICULOS CON SPRING BOOT

Kevin Andres Jimenez Casares

Facultad de Ingeniería y Tecnología, Fundación Universitaria Internacional de La Rioja

Desarrollo de aplicaciones web

Ernesto Solano Hernandez

11 de mayo del 2026

Índice

Índice	2
Introducción	3
Arquitectura general del sistema	4
Servidor Eureka	4
API Gateway	5
Comunicación entre microservicios	5
Microservicio Vehículos	6
GET /api/vehicles	6
GET /api/vehicles/{id}	6
POST /api/vehicles	7
PUT /api/vehicles/{id}	7
DELETE /api/vehicles/{id}	7
Microservicio Operaciones	8
GET /api/vehicles/available	8
POST /api/rentals	8
GET /api/rentals/{id}	9
GET /api/rentals	9
DELETE /api/rentals/{id}	9
Conclusiones	10
Referencias bibliográficas	10

Introducción

En este proyecto basado en la gestión de vehículos y operaciones de alquiler, permite realizar el ejercicio de implementar un sistema funcional aplicando conceptos importantes relacionados con APIs REST en el lenguaje de java con Spring Boot, conexión a la base de datos, arquitectura distribuida, registro de servicios, comunicación entre microservicios y despliegue mediante la herramienta de contenedores Docker que es muy usada en el mercado actual.

El sistema se desarrolló en arquitectura de microservicios utilizando Spring Boot y diferentes herramientas del ecosistema Spring Cloud. El objetivo principal fue construir una aplicación distribuida donde los servicios pudieran comunicarse entre sí de manera independiente, organizada y escalable para el futuro.

Durante el desarrollo de la aplicación se investigó y se implementaron diferentes componentes importantes como base de datos MySQL, JPA, Eureka Server, API Gateway y dos microservicios principales llamados “vehicle-service” y “management-vehicle”.

Cada componente cumple una función específica dentro de la arquitectura. También se utilizaron tecnologías de despliegue populares como Docker y Docker Compose para facilitar la ejecución local del sistema y permitir que todos los servicios puedan iniciar su correcto funcionamiento de manera automática utilizando un solo comando.

El proyecto permitió extender el conocimiento de las herramientas que se usan en el mercado realizándolo de forma práctica y conociendo cómo funcionan las arquitecturas modernas utilizadas actualmente en el desarrollo backend empresarial.

Arquitectura general del sistema

La arquitectura implementada sigue el modelo de microservicios, donde cada servicio funciona de forma independiente y se comunica mediante HTTP utilizando APIs REST.

Los componentes principales del sistema son:

- **Eureka Server:** Encargado del registro y descubrimiento de servicios.
- **API Gateway:** Punto único de entrada y redirección para todas las peticiones.
- **Vehicle-service:** Microservicio encargado de la gestión de vehículos.
- **Management-vehicle:** Microservicio encargado de operaciones y alquileres.
- **Docker Compose:** Utilizado para levantar todos los servicios.

La arquitectura escogida permite escalar servicios de forma independiente y mantener una mejor organización del código. Esto se realiza en busca de mantener la escalabilidad y el orden correspondientes entre los componentes que hacen parte.

Servidor Eureka

El servidor Eureka fue implementado utilizando Spring Cloud Netflix Eureka Server. Su función principal es actuar como registro central de servicios. Cuando un microservicio inicia, automáticamente se registra en Eureka indicando:

- *Nombre del servicio, Puerto, Dirección IP, Estado del servicio*

Gracias a esto, los servicios pueden encontrarse automáticamente sin necesidad de configurar direcciones manualmente.

API Gateway

El API Gateway cumple la función de ser como punto central de acceso al sistema. Todas las solicitudes realizadas por el cliente pasan primero por el Gateway, el cual redirige la petición al microservicio correspondiente siendo centralizada toda la información que ingresa. Esto ayuda a mantener una arquitectura más organizada, facilita futuras y las implementaciones relacionadas con seguridad, autenticación, entre otras son más escalables. El Gateway utiliza Eureka para encontrar automáticamente los microservicios registrados.

Comunicación entre microservicios

La comunicación entre los microservicios se realiza utilizando nombres registrados en Eureka. Esto significa que no es necesario utilizar localhost ni puertos específicos.

Por ejemplo:

<http://vehicle-service/api/vehicles>

Esto permite una arquitectura mucho más flexible y preparada para escalar. El microservicio de operaciones consulta información del servicio de vehículos antes de registrar un alquiler.

Microservicio Vehículos

El microservicio vehicle-service fue desarrollado para administrar toda la información relacionada con vehículos. Este servicio expone diferentes endpoints REST para realizar operaciones CRUD.

GET /api/vehicles

Obtiene todos los vehículos registrados.

Ejemplo de respuesta:

```
{
  "id": 1,
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123"
}
```

GET /api/vehicles/{id}

Obtiene un vehículo por ID.

Ejemplo de respuesta:

```
{
  "id": 1,
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123"
}
```

POST /api/vehicles

Crear un nuevo vehículo.

Ejemplo de body JSON:

```
{
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123",
  "available": true
}
```

PUT /api/vehicles/{id}

Actualiza un vehículo existente.

Ejemplo de body JSON:

```
{
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123",
  "available": true
}
```

DELETE /api/vehicles/{id}

Elimina un vehículo.

200 OK: Proceso finalizado sin problemas

400 Not found: Registros no encontrados

500 Internal server error: Error no controlado

Microservicio Operaciones

El microservicio management-vehicle fue desarrollado para administrar operaciones relacionadas con alquileres de vehículos. Este servicio también expone APIs REST y realiza comunicación con el microservicio de vehículos.

GET /api/vehicles/available

Obtiene vehículos disponibles.

Ejemplo de respuesta

```
{
  "brand": "Toyota",
  "model": "Corolla",
  "plate": "ABC123",
  "available": true
}
```

POST /api/rentals

Registra un alquiler.

Ejemplo de un body json:

```
{
  "vehicleId": 1,
  "customerName": "Kevin Jimenez",
  "days": 5
}
```


GET /api/rentals/{id}

Obtiene un alquiler por ID.

Ejemplo de respuesta

```
{
  "id": 10,
  "vehicleId": 1,
  "status": "ACTIVE"
}
```

GET /api/rentals

Obtiene todos los alquileres.

Ejemplo de respuesta

```
{
  "id": 10,
  "vehicleId": 1,
  "status": "ACTIVE"
}
```

DELETE /api/rentals/{id}

Cancela un alquiler.

200 OK: Proceso finalizado sin problemas

400 Not found: Registros no encontrados

500 Internal server error: Error no controlado

Conclusiones

Este proyecto permitió comprender mejor cómo funciona una arquitectura basada en microservicios y las ventajas que ofrece frente a una aplicación monolítica tradicional. Uno de los aspectos más interesantes fue trabajar con Eureka y entender cómo los servicios pueden encontrarse automáticamente sin necesidad de configurar direcciones manualmente. También fue bastante útil implementar un API Gateway, ya que ayuda a mantener una estructura más organizada y facilita futuras mejoras relacionadas con seguridad y administración de tráfico. Por otro lado, Docker ayudó mucho para simplificar la ejecución de todos los servicios, aunque al principio aparecieron algunos problemas relacionados con puertos y comunicación entre contenedores. En general, el proyecto permitió aplicar conocimientos prácticos sobre Spring Boot, APIs REST, Docker, Eureka y arquitectura distribuida, reforzando conceptos importantes del desarrollo backend moderno.

Referencias bibliográficas

Spring Boot Documentation. <https://spring.io/projects/spring-boot>

Spring Cloud Netflix Documentation. <https://spring.io/projects/spring-cloud-netflix>

Docker Documentation. <https://docs.docker.com/>

Oracle Java Documentation. <https://docs.oracle.com/en/java/>

Chris Richardson. Microservices Patterns. Manning Publications.